

TP arraylist

Matisse

kra

Sommaire

Objectif du TP	2
A. Le Modèle : La classe Livre	2
B. Le Stockage : ArrayList	3
C. La Vue : Interface Swing (Accueil.java)	4
Fonctionnalités développées	4
Difficultés rencontrées et Solutions	5
Conclusion	5

Objectif du TP

Le but de ce travail pratique était de développer une application de gestion de bibliothèque. Nous devons passer d'une simple exécution en console à une interface graphique (Application Window) , capable de stocker des livres, d'en ajouter de nouveaux et de rechercher des informations par ISBN.

Le Modèle : La classe Livre

Nous avons créé une classe représentant l'objet métier avec les caractéristiques suivantes :

A) Les Attributs C'est la partie qui définit les données que le livre contient.

- `private String ISBN;`
- `private String titre;`
- `private String Auteur;`
- `private int prix;`
- `private` C'est le principe d'encapsulation. Cela signifie que personne (pas même la fenêtre Accueil) ne peut modifier ces variables directement. C'est une sécurité : on protège les données internes de l'objet.
- Le Types : On utilise `String` pour du texte et `int` pour le prix (comme demandé dans le sujet).

B) Le Constructeur C'est la méthode qui porte le même nom que la classe. Elle est appelée une seule fois par livre : au moment de sa création avec le mot-clé `new`

- `public Livre (String ISBN, String titre, String auteur, int prix) {`
- `this.ISBN = ISBN;`
- `this.titre = titre;`
- `this.Auteur = auteur;`
- `this.prix = prix;`
- `}`
- Son rôle : Il force le programmeur à donner toutes les infos obligatoires dès le début. Tu ne peux pas créer un "Livre vide".

C) Les Accesseurs ou "Getters"

- Comme les attributs sont `private` (verrouillés), on a besoin de "portes" pour voir ce qu'il y a dedans.
 - `public String getISBN() { return ISBN; }`
 - `public String getTitre() { return titre; }`
 - `public int getPrix() { return prix; }`
- Dans la fenêtre Accueil, quand on clique sur "Rechercher", on a besoin de vérifier si l'ISBN saisi correspond à celui du livre. Sans `getISBN()`, la fenêtre n'a pas le droit de lire la variable `ISBN` du livre.

```
1 package liste;
2 public class Livre {
3
4     // Attributs privés
5     private String ISBN;    Rename this field "ISBN" to match the regular expressio
6     private String titre;
7     private String auteur;    Field auteur can be final
8     private int prix;
9
10    // Constructeur
11    public Livre(String ISBN, String titre, String auteur, int prix) {    Rename th
12        this.ISBN = ISBN;
13        this.titre = titre;
14        this.auteur = auteur;
15        this.prix = prix;
16    }
17
18    // --- LES GETTERS ---
19
20    public String getISBN() {
21        return ISBN;
22    }
23
24    public String getTitre() {
25        return titre;
26    }
27
28    public String getAuteur() {
29        return auteur;
30    }
31
32    public int getPrix() {
33        return prix;
34    }
35
36    // --- LES SETTERS---
37    public void setISBN(String ISBN) { this.ISBN = ISBN; }    Rename this local var
38    public void setTitre(String titre) { this.titre = titre; }
39    public void setPrix(int prix) { this.prix = prix; }
40
41    // Méthode Afficher
42    public void afficherLivre() {
43        System.out.println("Livre: " + titre + " (" + ISBN + ") - " + prix + "€");
44    }
45 }
```

B. Le Stockage : ArrayList

Au lieu d'utiliser un tableau statique, nous avons utilisé une collection ArrayList<Livre>.

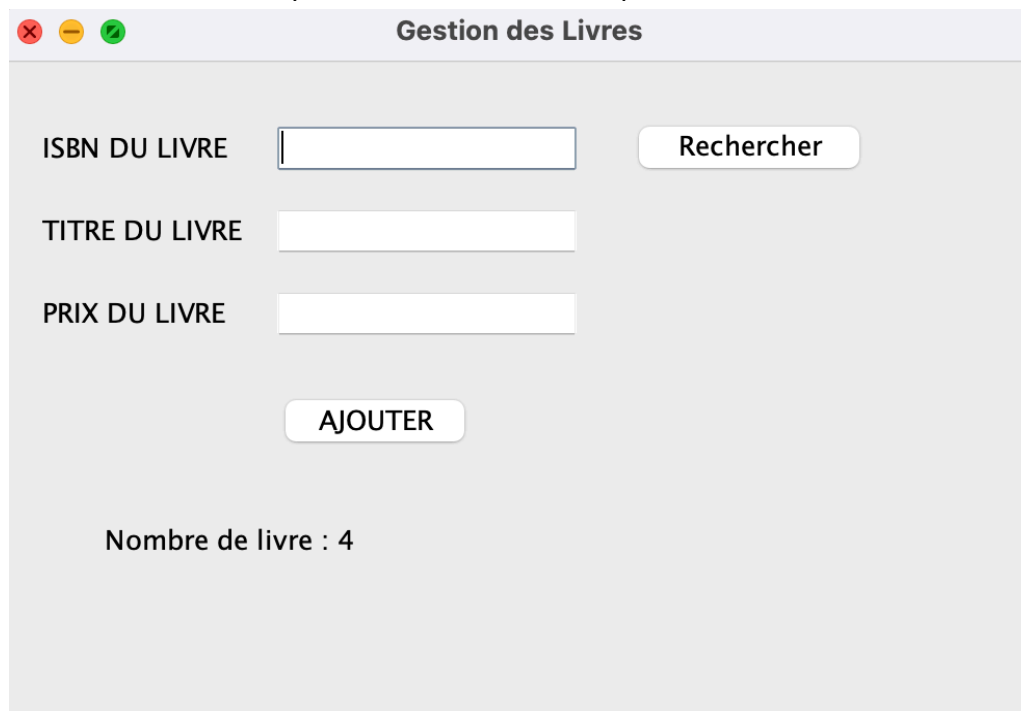
- Avantage : Cette structure est dynamique (pas de taille fixe), ce qui permet d'ajouter des livres indéfiniment via la méthode .add().
- Initialisation : Une liste contenant 4 livres par défaut est chargée au lancement de l'application.

```
1 package liste;
2 import java.util.ArrayList;
3 import java.util.List;
4
5 public class Main {
6     Run main | Debug main | Run | Debug
7     public static void main(String[] args) {
8         Livre livre1;
9         Livre livre2;
10        int prix;    Variable prix is neither read or written to
11        String titre, auteur, isbn;    Variable titre is neither read or written to
12        livre1 = new Livre(ISBN: "11111", titre: "Titre 1", auteur: "auteur 1", prix: 1);
13        livre2 = new Livre(ISBN: "22222", titre: "Titre 2", auteur: "auteur 2", prix: 2);
14
15        List<Livre> bibliotheque = creerGroupe(livre1, livre2);
16        System.out.println(" taille de la bibliotheque: " + bibliotheque.size());    Replace this use
17        for (Livre l : bibliotheque)
18            l.afficherLivre();
19    }
20
21    public static List<Livre> creerGroupe(Livre l1, Livre l2) {
22        // On initialise la collection (vide au départ)
23        // Les chevrons <Livre> forcent la liste à ne contenir QUE des livres
24        List<Livre> listeDesLivres = new ArrayList<>();
25        // On ajoute les paramètres à la collection
26        listeDesLivres.add(l1);
27        listeDesLivres.add(l2);
28        listeDesLivres.add(new Livre(ISBN: "33333", titre: "Titre 3", auteur: "auteur 3", prix: 3));
29        listeDesLivres.add(new Livre(ISBN: "44444", titre: "Titre 4", auteur: "auteur 4", prix: 4));
30        // On retourne la collection remplie
31        return listeDesLivres;
32    }
33 }
34 }
```

C. La Vue : Interface Swing (Accueil.java)

L'interface utilisateur a été conçue avec la bibliothèque Java Swing. Contrairement à l'outil "WindowBuilder" suggéré pour Eclipse, l'interface a été codée manuellement sous VS Code:

- Conteneur : Utilisation d'un JPanel avec un layout null pour un positionnement absolu (setBounds) des éléments.
- Composants :
 - JTextField pour la saisie (ISBN, Titre, Prix).
 - JButton pour les actions (Ajouter, Rechercher).
 - JLabel pour les libellés et le compteur de livres.



Fonctionnalités développées

Fonction "Ajouter"

Un bouton permet de récupérer les données saisies par l'utilisateur.

- Conversion de type : Le prix étant saisi en texte (String), nous avons utilisé Integer.parseInt() pour le convertir en entier (int) avant de créer l'objet.
- Mise à jour : Le livre est ajouté à l'ArrayList et le label affichant le "Nombre de livres" est actualisé dynamiquement.

Fonction "Rechercher"

Cette fonction parcourt l'ArrayList pour retrouver un livre via son ISBN.

- Algorithme : Une boucle for-each traverse la liste. Si livre.getISBN() correspond à la saisie, les champs Titre et Prix sont remplis.
- Gestion d'erreur : Si la boucle se termine sans résultat, le mot "ERREUR" est affiché dans les champs correspondants.

Difficultés rencontrées et Solutions

Durant la réalisation du TP sur l'environnement macOS / VS Code, plusieurs problèmes techniques ont été résolus :

1. Méthode manquante : Une erreur sur getISBN() est survenue car l'accessor n'était pas défini dans la classe Livre. La méthode a été ajoutée pour permettre la comparaison dans la fonction de recherche.

Conclusion

Ce TP a permis de consolider la compréhension de la POO (Programmation Orientée Objet) en Java, notamment l'interaction entre une classe métier et une interface graphique. L'utilisation des ArrayList a démontré l'intérêt des collections dynamiques par rapport aux tableaux classiques.